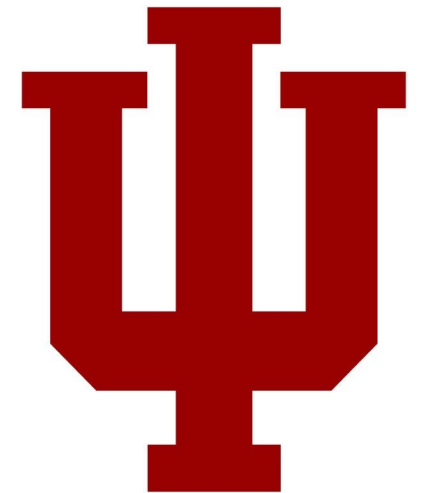


06: Memory-Mapped I/O

Engr 315: Hardware / Software Codesign
Andrew Lukefahr
Indiana University

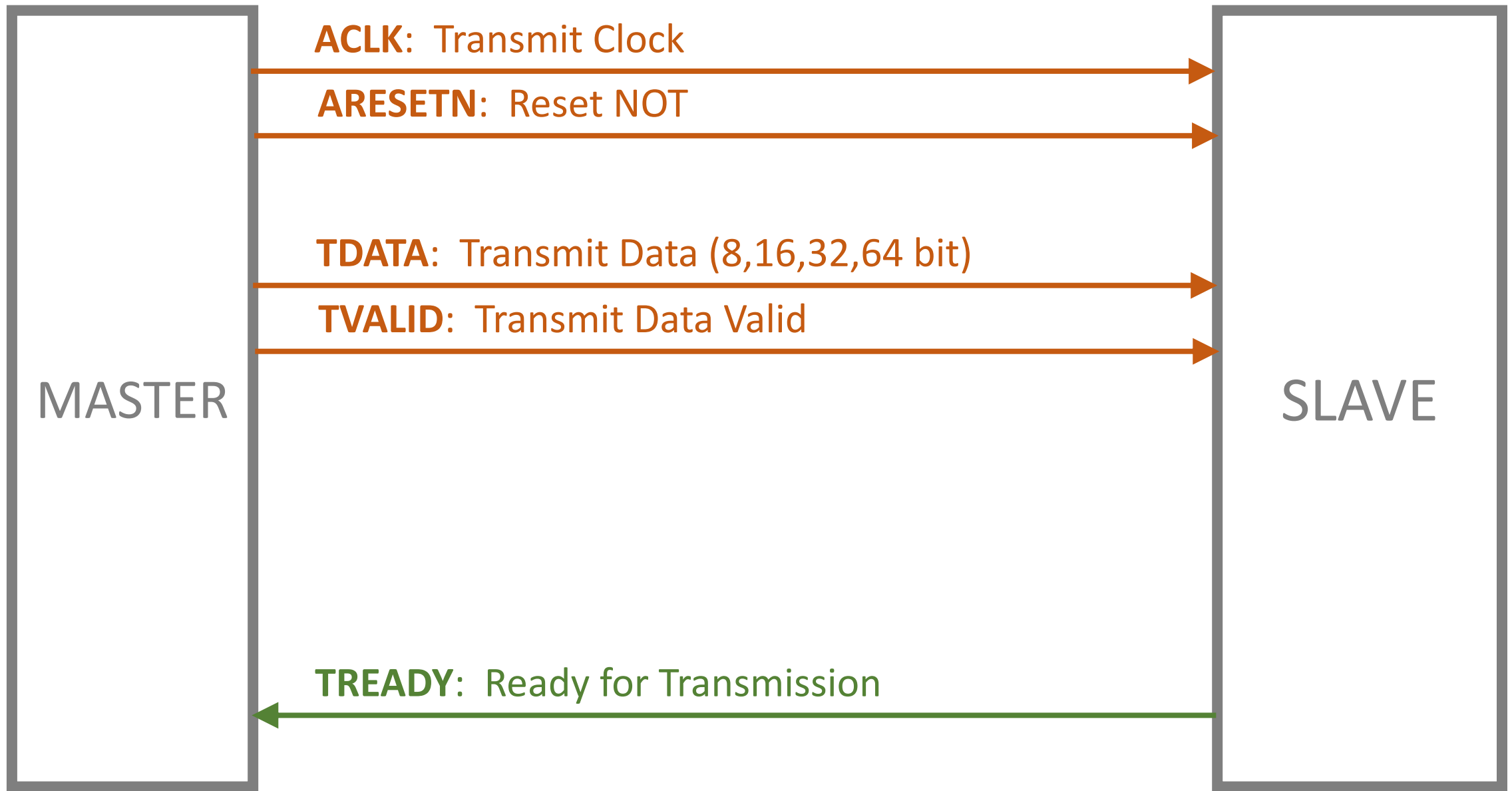


Announcements

- P2: Due Tonight
 - Need a Pynq
 - Groups of 2 allowed
- P3: Out now!
 - Due next Friday

Optimizations thus far

- Algorithmic complexity
- Removing redundant computation
- ~~Multithreading / Multiprocessing*~~
- Python/C/Asm Interfacing
- **Map to Hardware**



AXI Stream Interface

Data (**TDATA**) is only transferred when

TVALID is 1.

This indicates the **MASTER** is trying to transmit new data.

TREADY is 1.

This indicates the **SLAVE** is ready to receive the data.

If either **TVALID** or **TREADY** are 0, no data is transmitted.

If **TVALID** and **TREADY** are 1, **TDATA** is transmitted

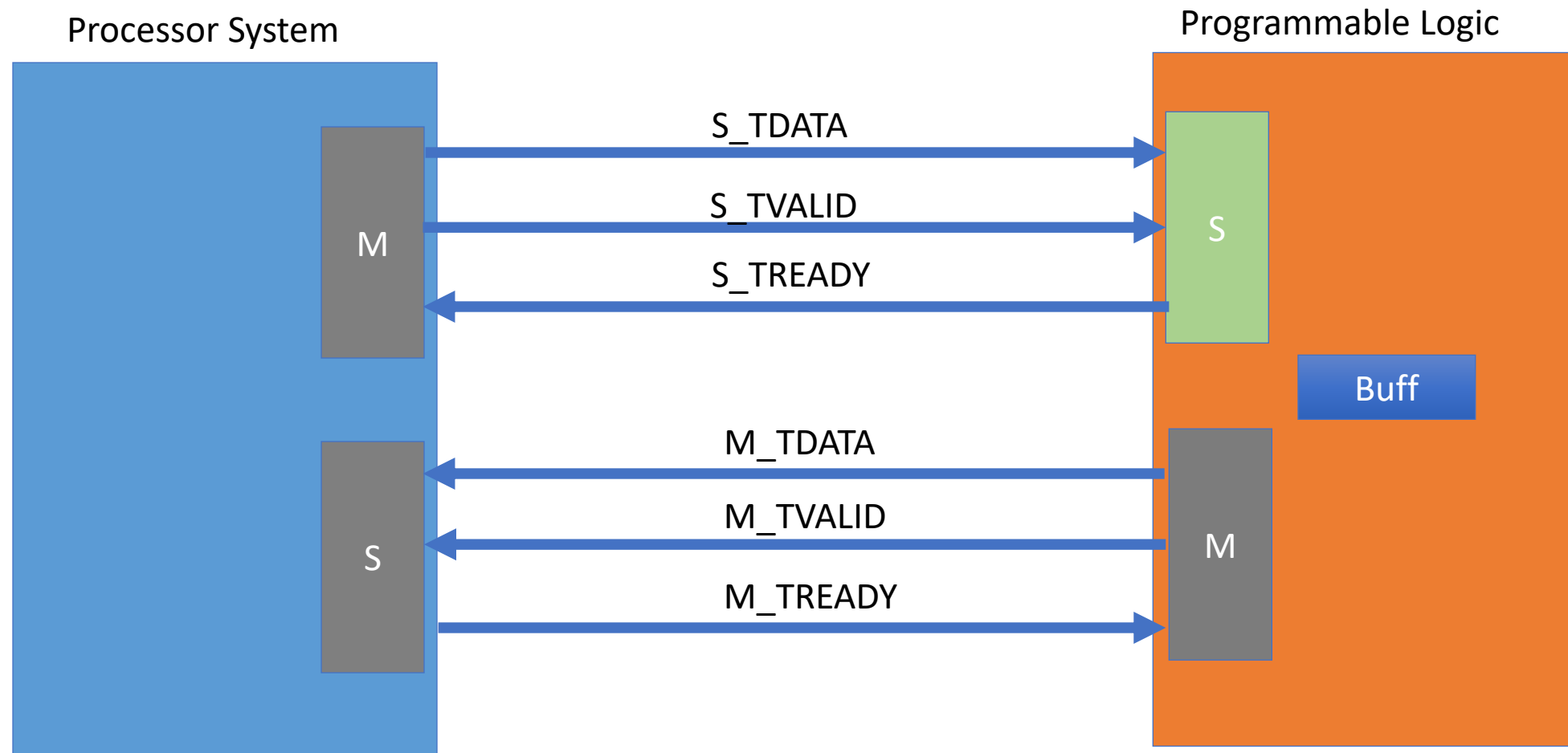
at the positive edge of **ACLK**

P3 maps EMA to hardware

```
import cEMA  
print (cEMA.cEMA(0))
```

```
import hwEMA  
print (hwEMA.hwEMA(0))
```

Block Diagram (Two AXI Stream Buses)

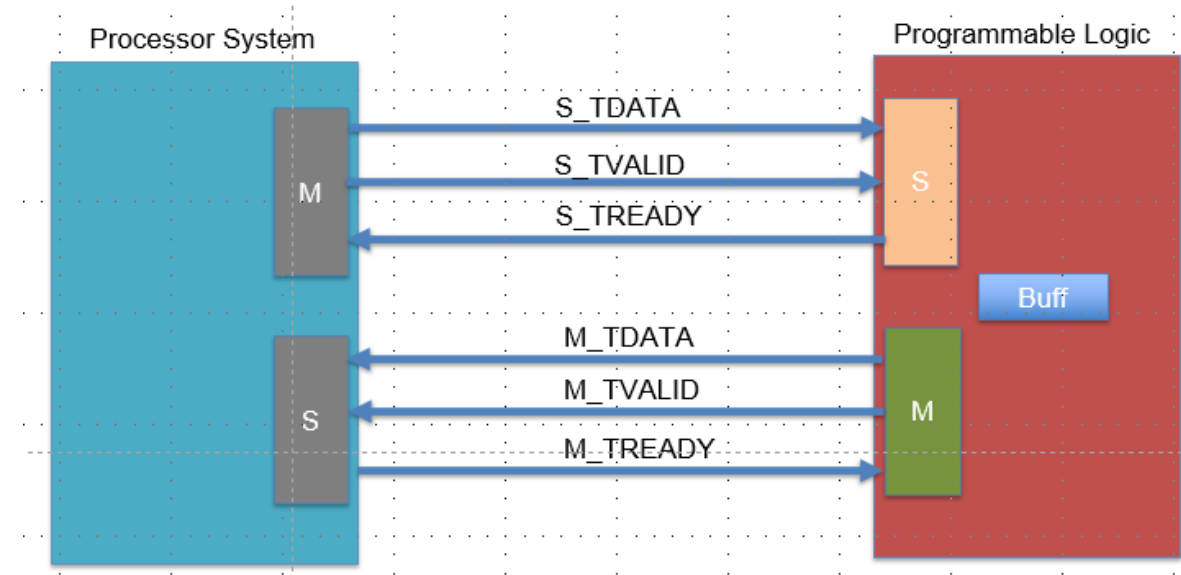


All Signal Names are from the perspective of the programmable logic. Recall that we are only designing the hardware for the PL side of the system



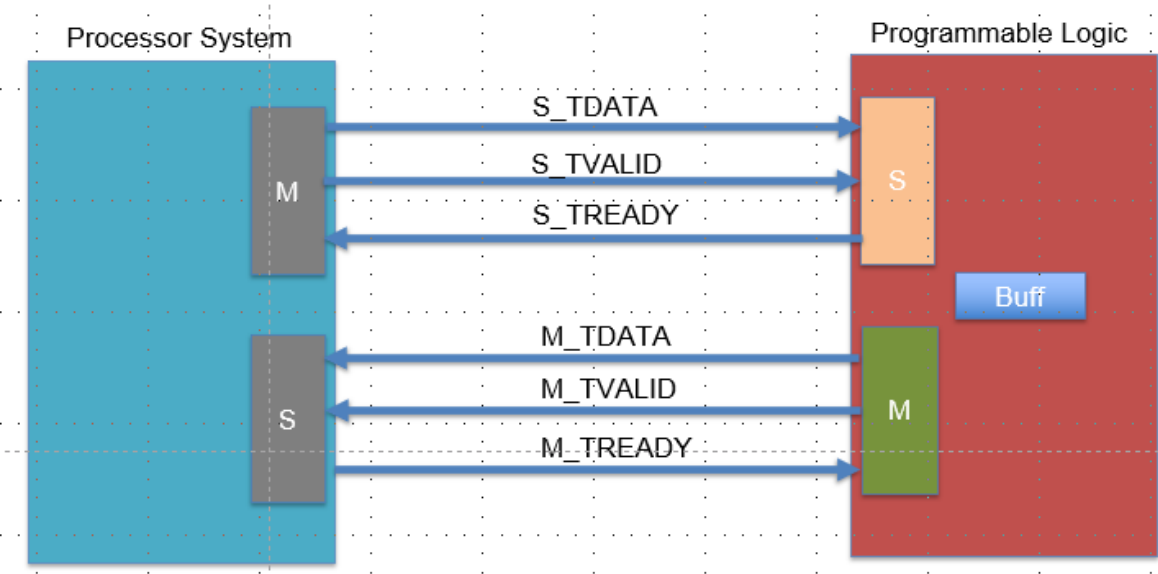
Buffer State Machine

```
module custom_hw_buf (  
    input ACLK,  
    input ARESETN,  
    input [31:0] S_TDATA,  
    input S_TVALID,  
    output logic S_TREADY,  
    output logic [31:0] M_TDATA,  
    output logic M_TVALID,  
    input M_TREADY  
);
```



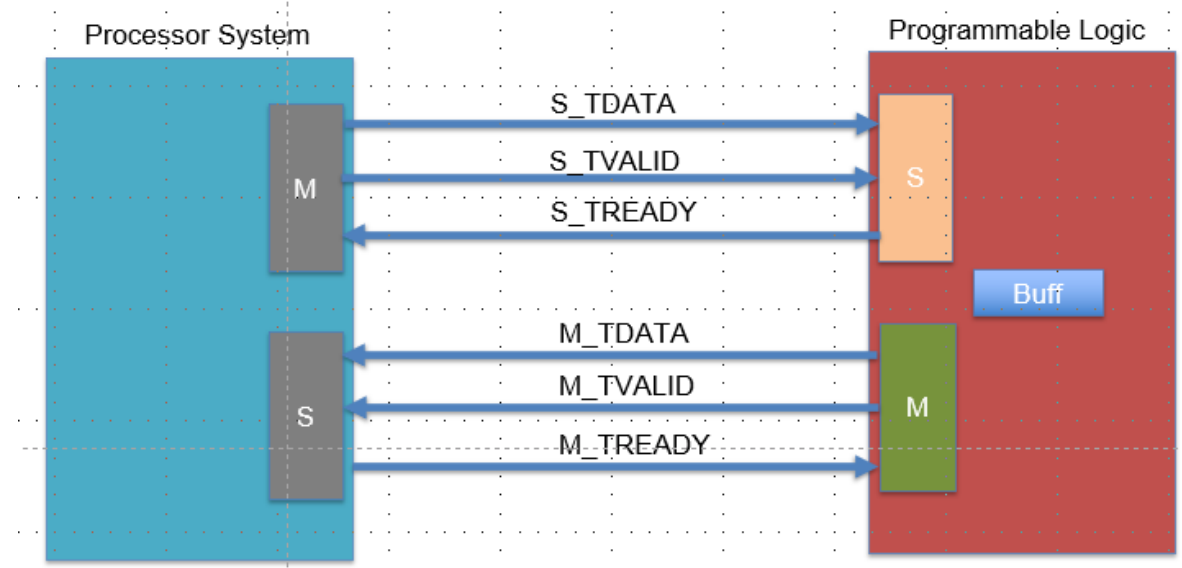
Buffer State Machine (Sequential Logic)

```
enum {S0, S1} state, nstate;  
logic [31:0] buff, nbuff;  
  
always_ff @(posedge ACLK) begin  
    if (~ARESETN) begin  
        state <= S0;  
        buff <= 32'h0;  
    end  
    else begin  
        state <= nstate;  
        buff <= nbuff;  
    end  
end
```



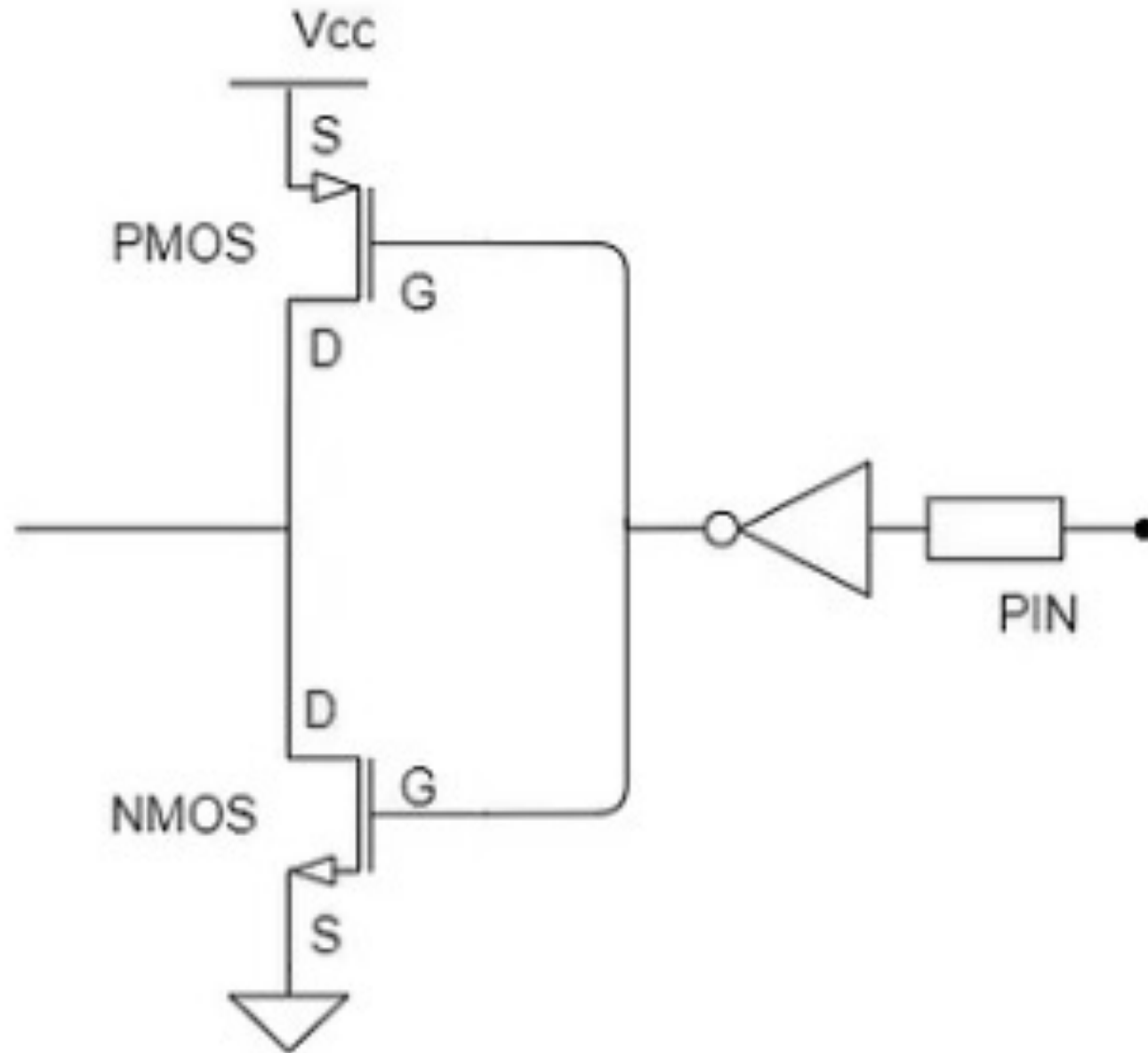
Buffer State Machine (Combinational Logic)

```
always_comb begin
    S_TREADY = 'h0;
    M_TDATA = 'h0;
    M_TVALID = 'h0;
    nstate = state;
    nbuff = buff;
    case (state)
        S0: begin
            M_TVALID = 'h0;
            S_TREADY = 'h1;
            if (S_TVALID) begin
                nstate = S1;
                nbuff = S_TDATA;
            end
        end
        S1: begin
            M_TDATA = nbuff;
            S_TREADY = 'h0;
            M_TVALID = 'h1;
            if (M_TREADY) begin
                nextState = S0;
            end
        end
    endcase // case (state)
end // always_comb
```



Inputs and Outputs

Input

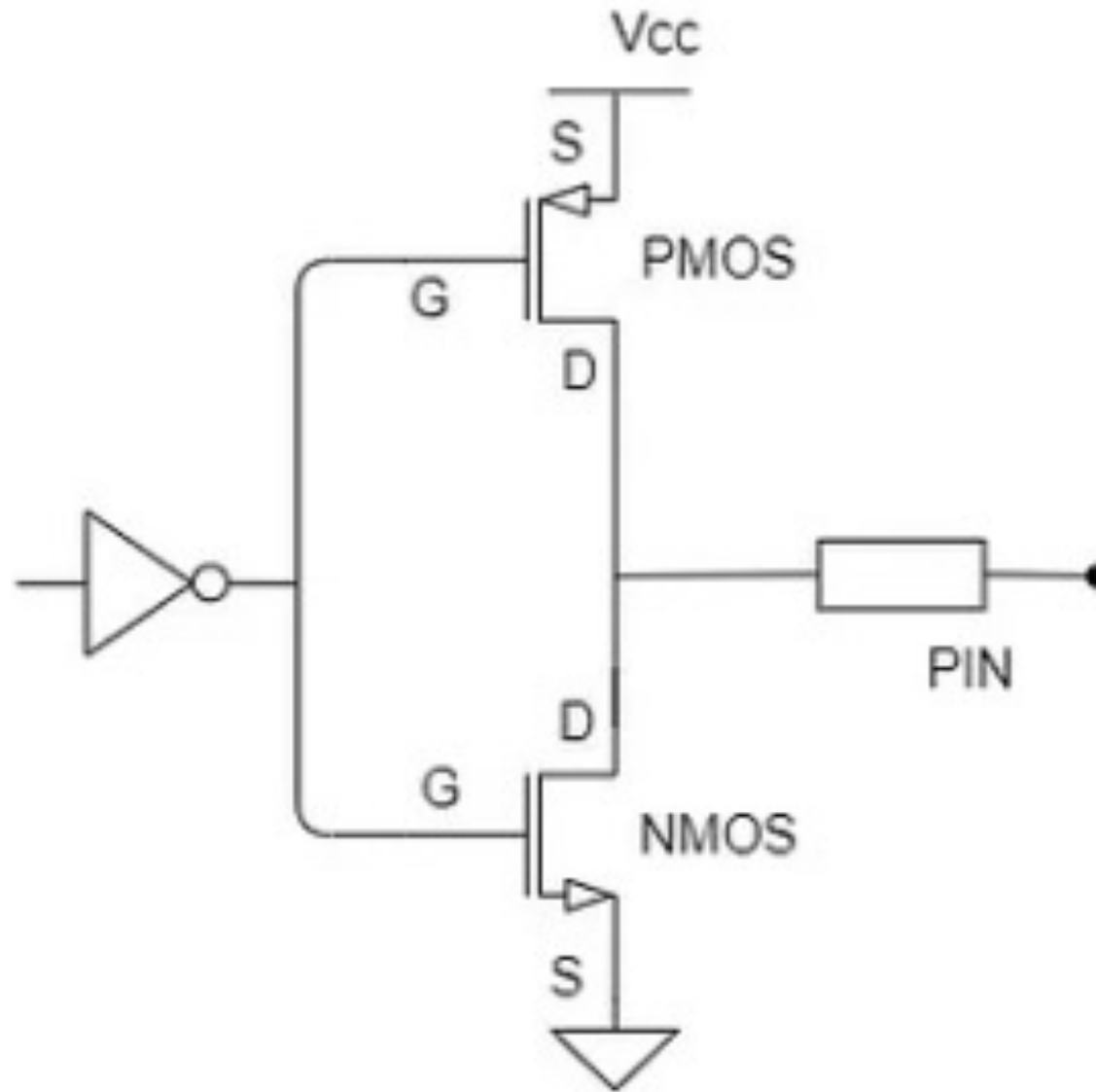


<https://suketushah.medium.com/gpio-as-fast-as-possible-f78cbb9db50e>



Inputs and Outputs

Output



<https://suketushah.medium.com/gpio-as-fast-as-possible-f78cbb9db50e>



How can we control the I/O from the CPU?

1. Direct Registers ?

2. Memory Mapping

- Port Mapping (x86 legacy): special instructions (IN, OUT)
- Memory Mapping: load/store instructions



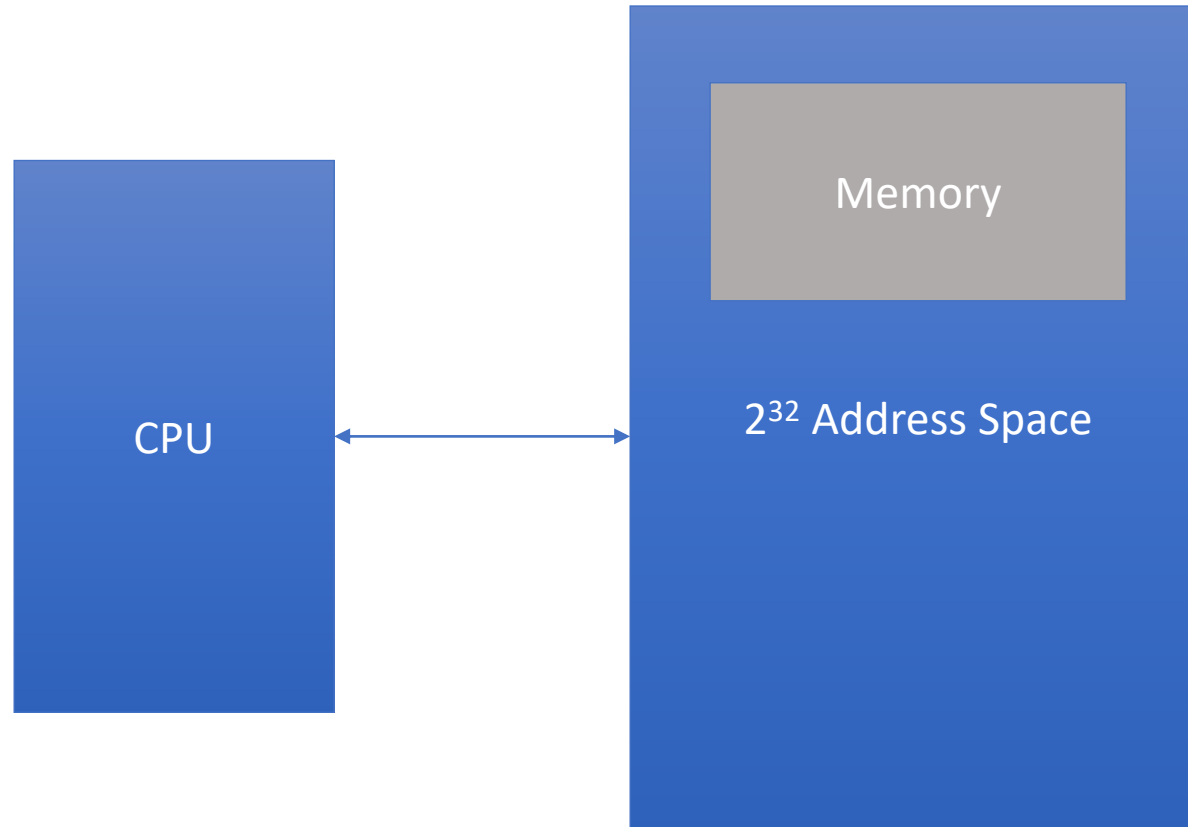
The Input/Output Problem: Bad Solution

The Input/Output Problem

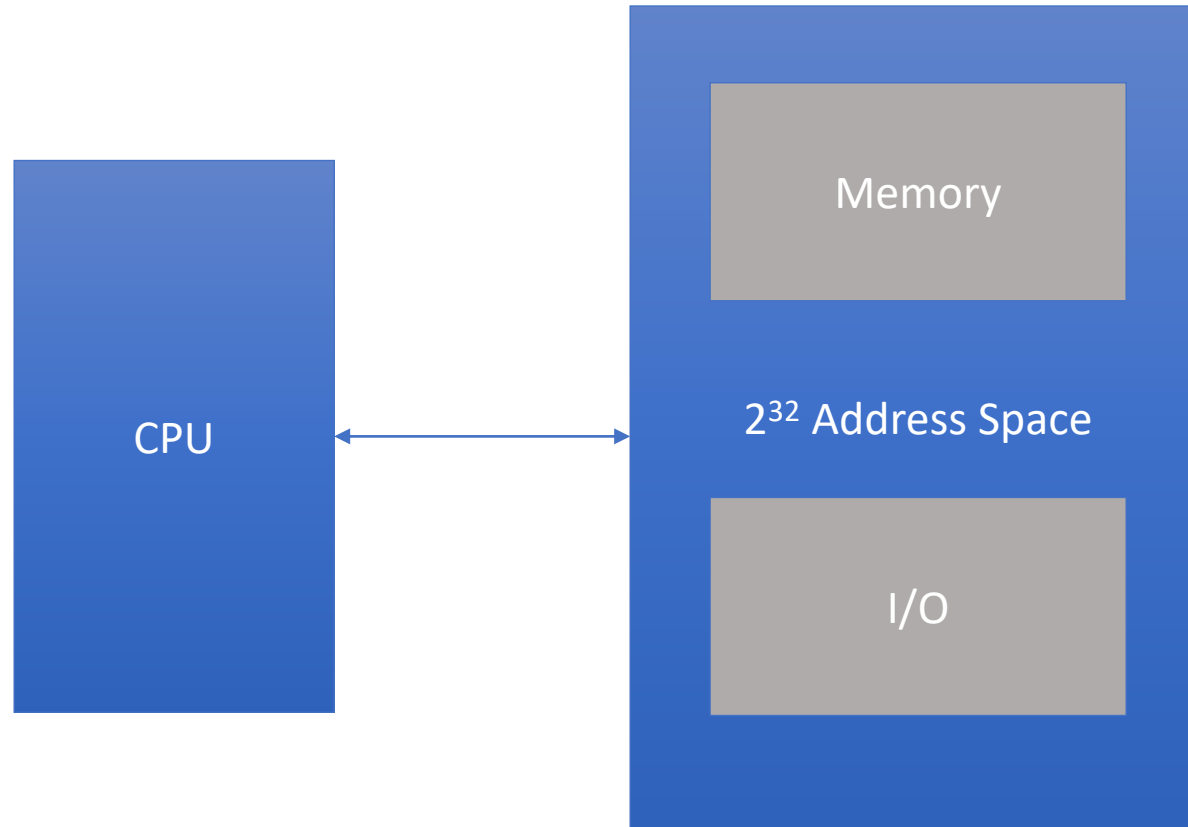
Memory-Mapped Inputs / Outputs

- Connect I/O to memory address
- “Pretend Memory” I/O accessed with native CPU load/store instructions

Memory Mapped I/O

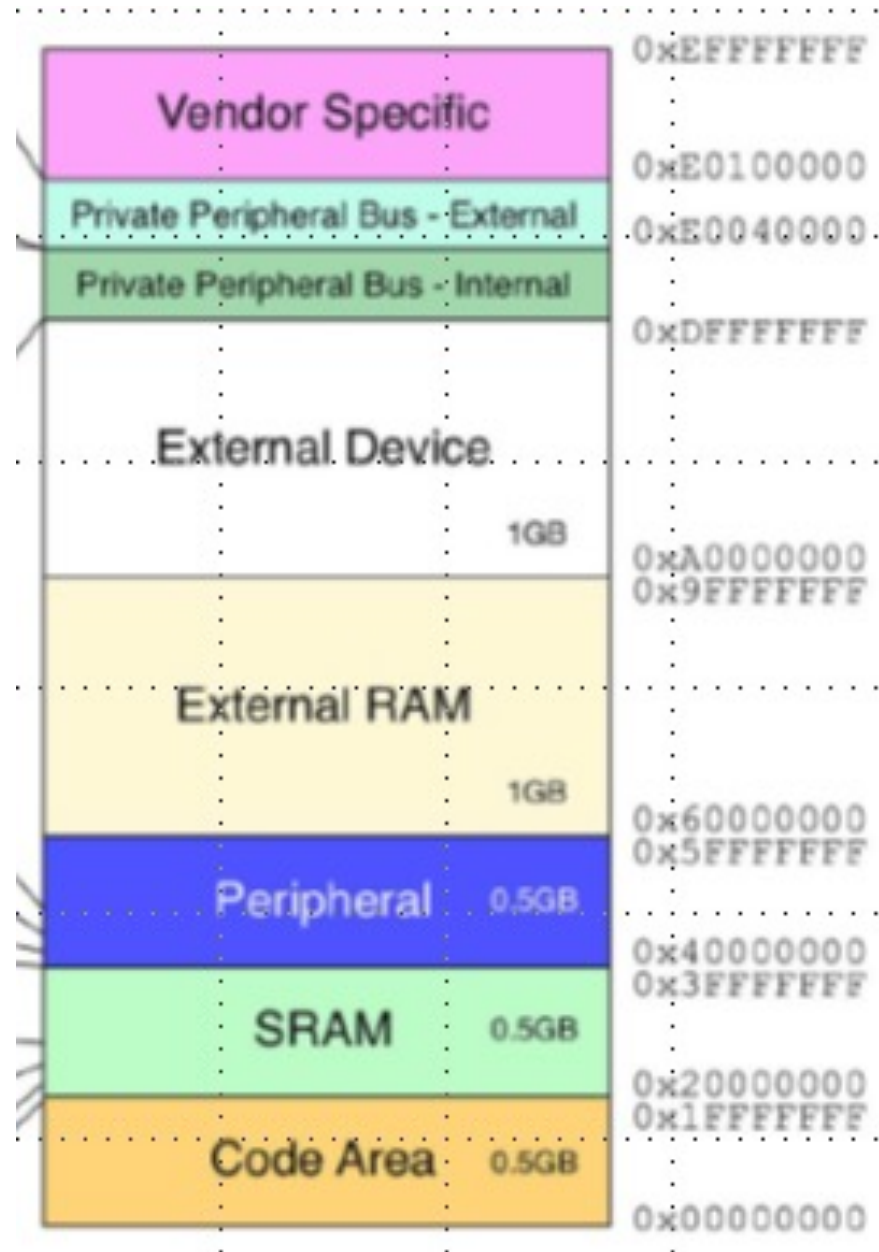


Memory Mapped I/O



Memory Mapped I/O

Embedded Systems Memory Example



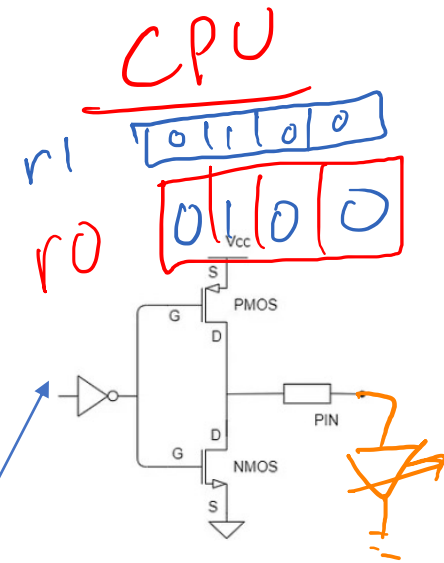
CPU + Memory

Small Memory Example

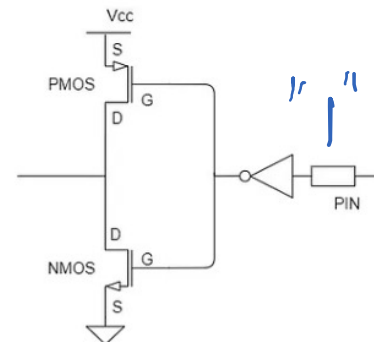
1. Store 0b0010 to 0x01
2. Load from 0x03 into r0 ✓
 - 0b0100 is read into r0
3. Store 0b0001 to 0x04
4. Load from 0x05 into r1

Memory Space

0x00				
0x01	0	0	1	0
0x02				
0x03	0	1	0	0
0x04	0	0	0	1
0x05	0	1	0	0



I/O



Memory-Mapped I/O

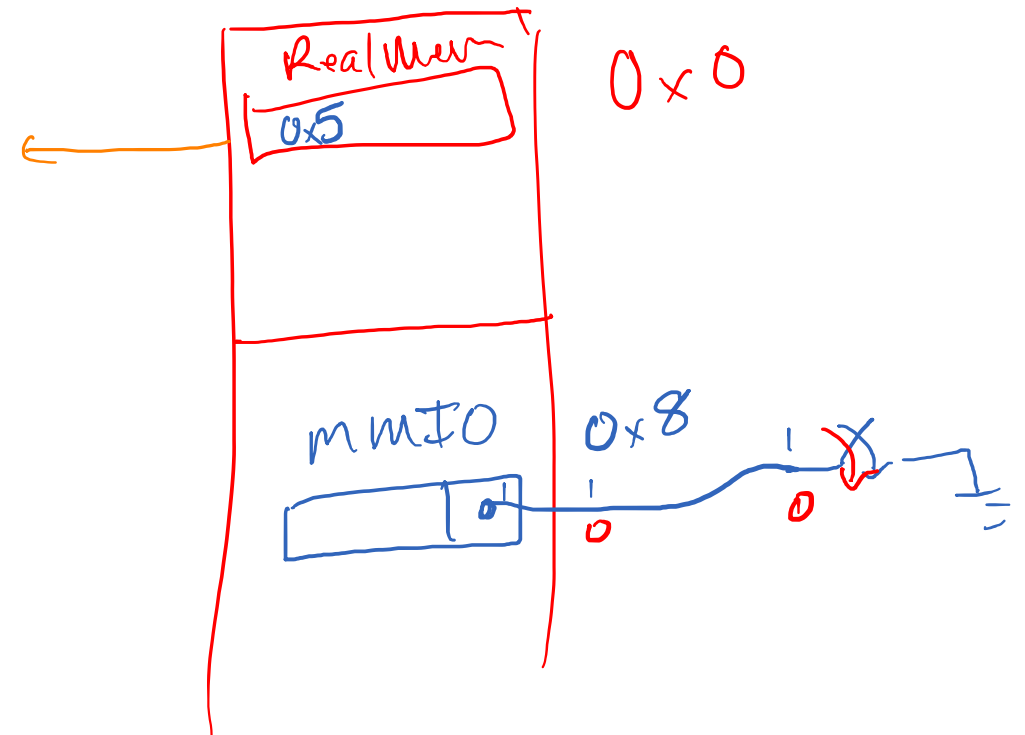
Goal: Connect I/O to memory address

store 0x5 → 0x0

load 0x0
load 0x0

store 0x2 → 0x8

load 0x8
load 0x8



MMIO from Assembly

- First, we need to see ARM assembly...

ARM Registers

ARM	Description	x86
R0	General Purpose	EAX
R1-R5	General Purpose	EBX, ECX, EDX, ESI, EDI
R6-R10	General Purpose	–
R11 (FP)	Frame Pointer	EBP
R12	Intra Procedural Call	–
R13 (SP)	Stack Pointer	ESP
R14 (LR)	Link Register	–
R15 (PC)	<- Program Counter / Instruction Pointer ->	EIP
CPSR	Current Program State Register/Flags	EFLAGS

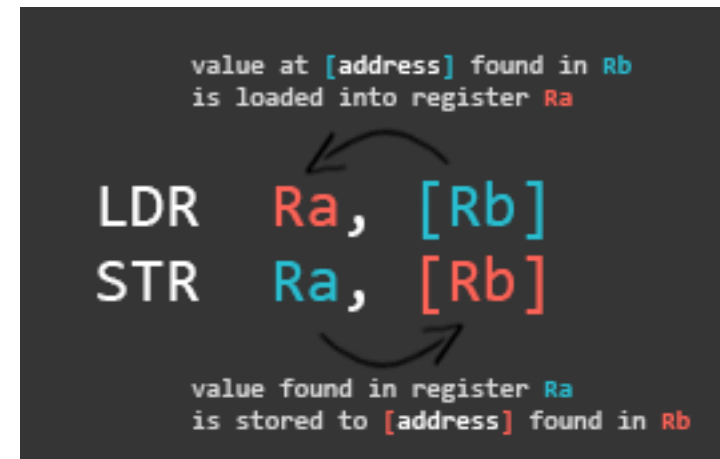
ARM Instructions

Instruction	Description	Instruction	Description
MOV	Move data	EOR	Bitwise XOR
MVN	Move and negate	LDR	Load
ADD	Addition	STR	Store
SUB	Subtraction	LDM	Load Multiple
MUL	Multiplication	STM	Store Multiple
LSL	Logical Shift Left	PUSH	Push on Stack
LSR	Logical Shift Right	POP	Pop off Stack
ASR	Arithmetic Shift Right	B	Branch
ROR	Rotate Right	BL	Branch with Link
CMP	Compare	BX	Branch and eXchange
AND	Bitwise AND	BLX	Branch with Link and eXchange
ORR	Bitwise OR	SWI/SVC	System Call

ARM Load + Store

LDR R2, [R0] @ [R0] - origin address is the value found in R0.

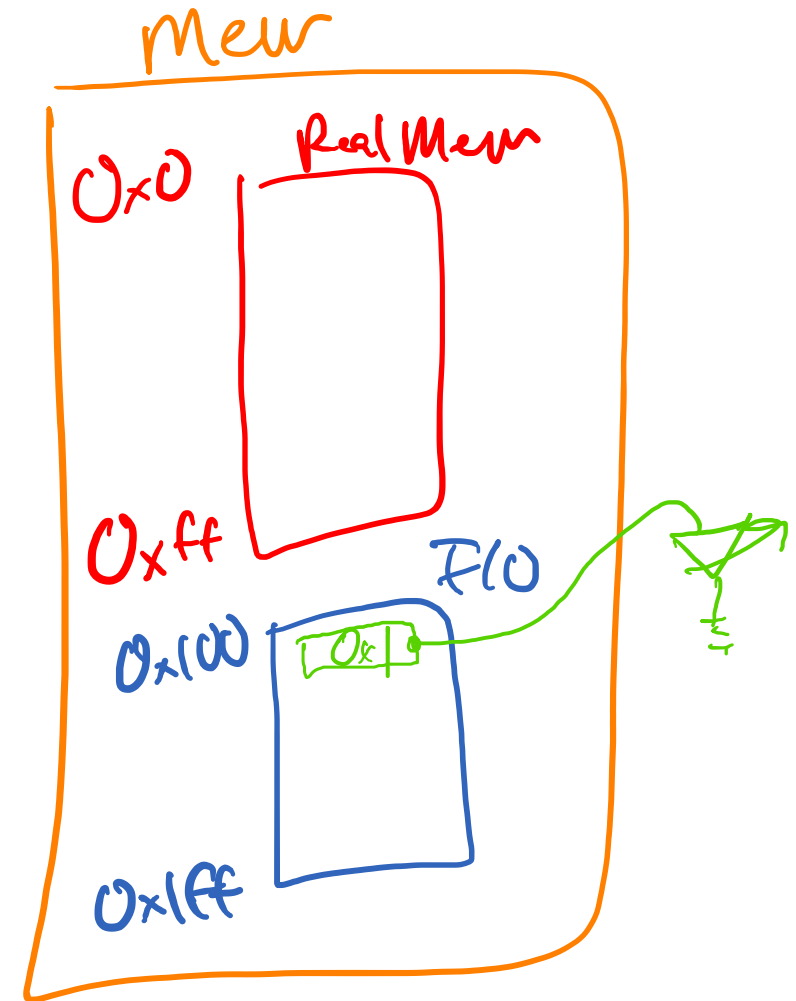
STR R2, [R1] @ [R1] - destination address is the value found in R1.



MMIO Store from ASM (ARM)

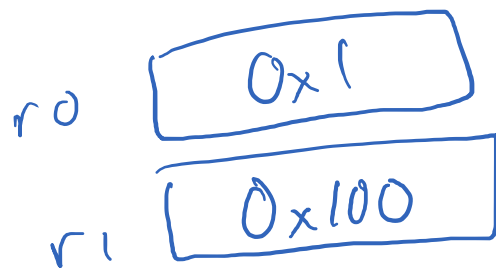
✓ mov r0, 0x1
✓ mov r1, 0x100
✓ str r0, [r1]
 ↑ ↑
 value mem address

CPU
r0: 0x1
r1: 0x100

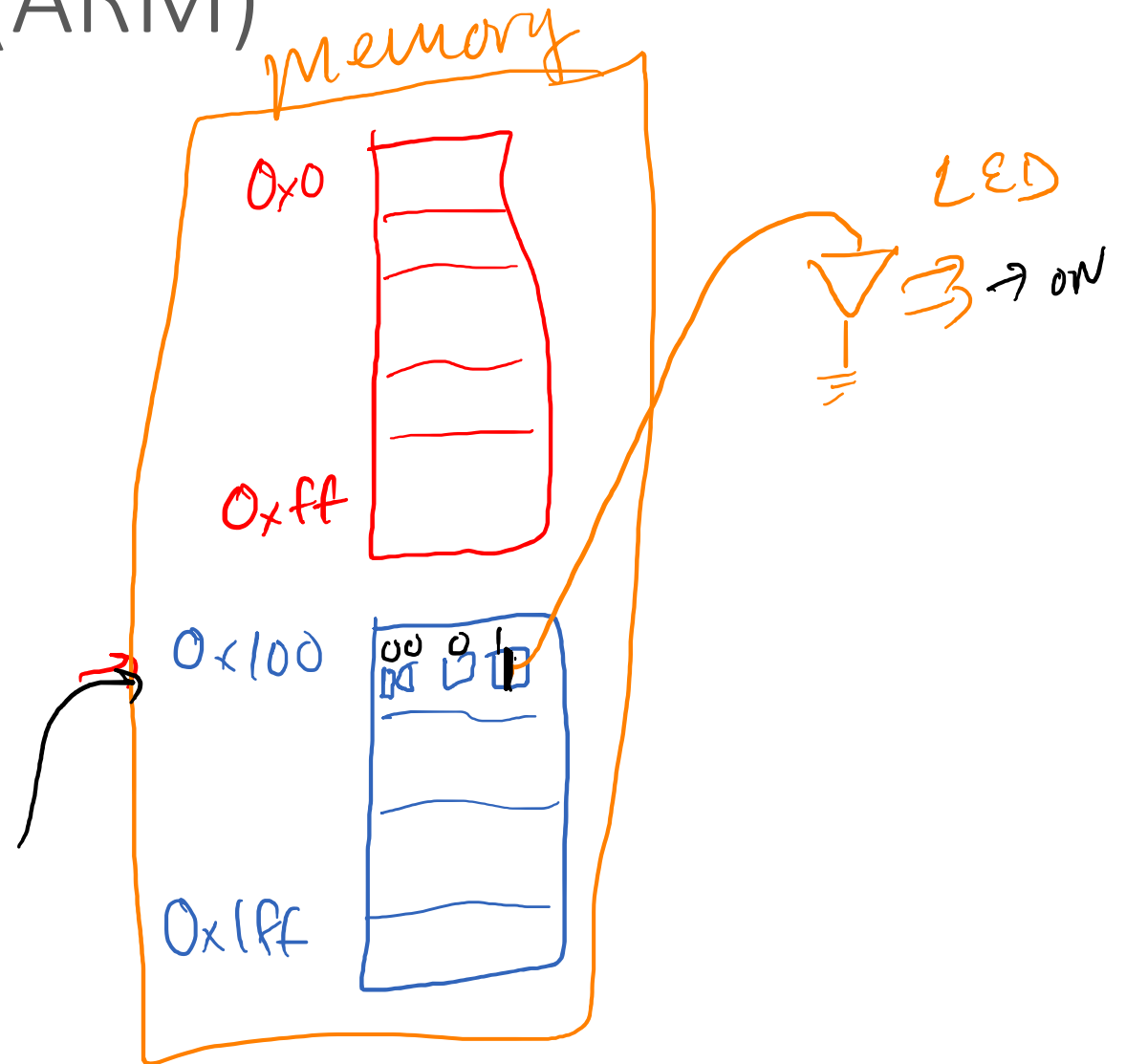


MMIO Store from ASM (ARM)

```
mov r0, 0x1  
mov r1, 0x100  
str r0, [r1]
```



Store `0x1` to `mem[0x100]`



MMIO Store from C

```
#define LED_ADDR 0x100
uint32_t * LED_REG = (uint32_t *) (LED_ADDR);
*LED_REG = 0x1;
```

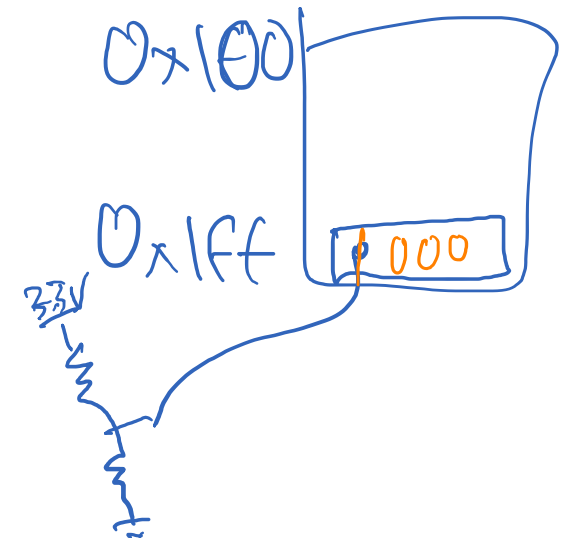
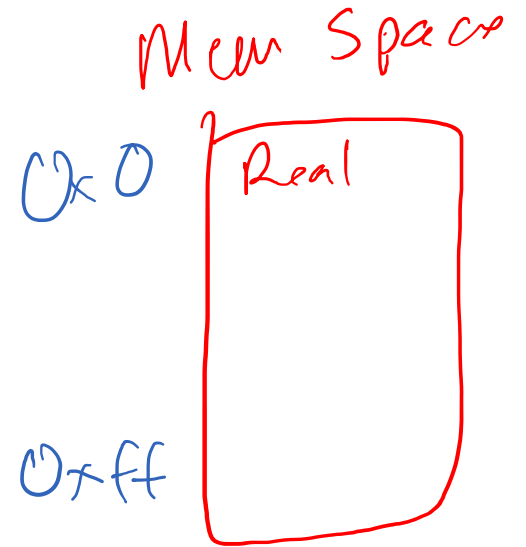
number ↓

↑ *pointer to a uint32_t*

MMIO Load from ASM (ARM)

```
mov r0, 0x1ff
ldr r1, [r0]
```

$r0 = 0x1ff$
 $r1 = 0x1000$



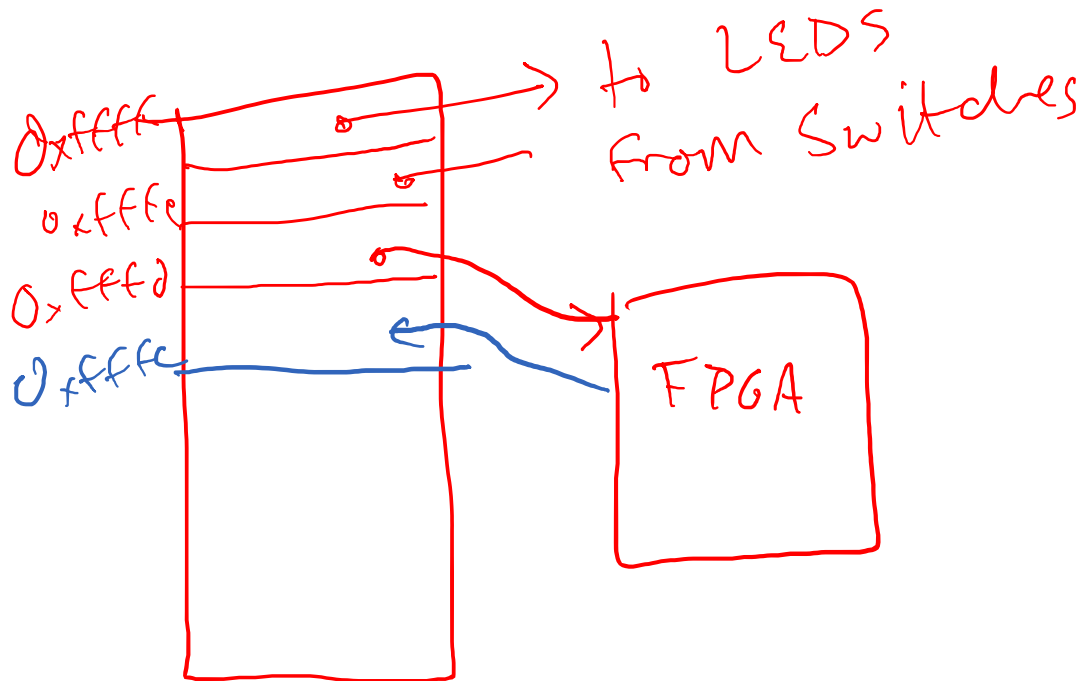
MMIO Load from C

```
#define SW_ADDR 0x1ff
uint32_t * SW_REG = (uint32_t *) (SW_ADDR);
int y = (*SW_REG);
```

MMIO for the FPGA

Memory-Mapped I/O

- I/O devices pretend to be memory
- Devices accessed with native CPU load/store instructions



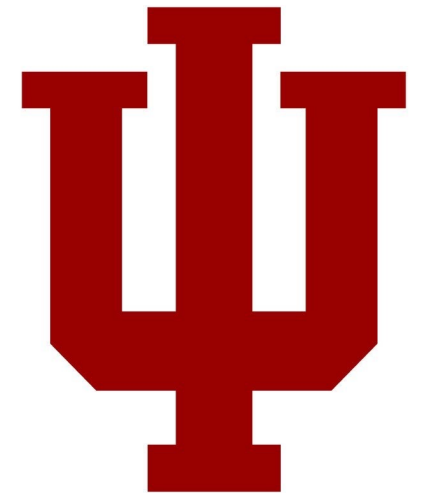
Question: What does this code do?

```
int y = 0;

int quit = y;
while(!quit)
{
    //more code
    quit = y;
}
```

06: Memory-Mapped I/O

Engr 315: Hardware / Software Codesign
Andrew Lukefahr
Indiana University



Problem: Does `quit` ever change here?
Do I need to recompute `(!quit)`?
(-O3 edition)

```
int y = 0;

int quit = y;
while(!quit)
{
    //more code
    quit = y;
}
```

Problem: The compiler is “helping”.

(-O3 edition)

```
int y = 0;

int quit = y;
while(!quit)
{
    //more code
    quit = y;
}
```

```
int y = 0;

int quit = y;
while(1)
{
    //more code
    quit = y;
}
```

What's the difference here?

```
int y = 0;

int quit = y;
while(!quit)
{
    //your code
    quit =y ;
}
```

```
int y = 0;
uint32_t * SW_REG = &y;

int quit = (*SW_REG);
while(!quit)
{
    //your code
    quit = (*SW_REG);
}
```

volatile Variables

- `volatile` keyword tells compiler that the memory value is subject to change randomly.
- Use `volatile` for all MMIO memory. The values change randomly!

Use `volatile` for
all MMIO memory.

(hint: P4)

What happens here?

```
#include <stdio.h>
#include <inttypes.h>

#define REG_FOO 0x40000140

int main () {
    volatile uint32_t *reg = (uint32_t *) (REG_FOO);
    *reg += 3;

    printf("0x%x\n", *reg); // Prints out new value
}
```

Let's find out...

```
vi test.c
```

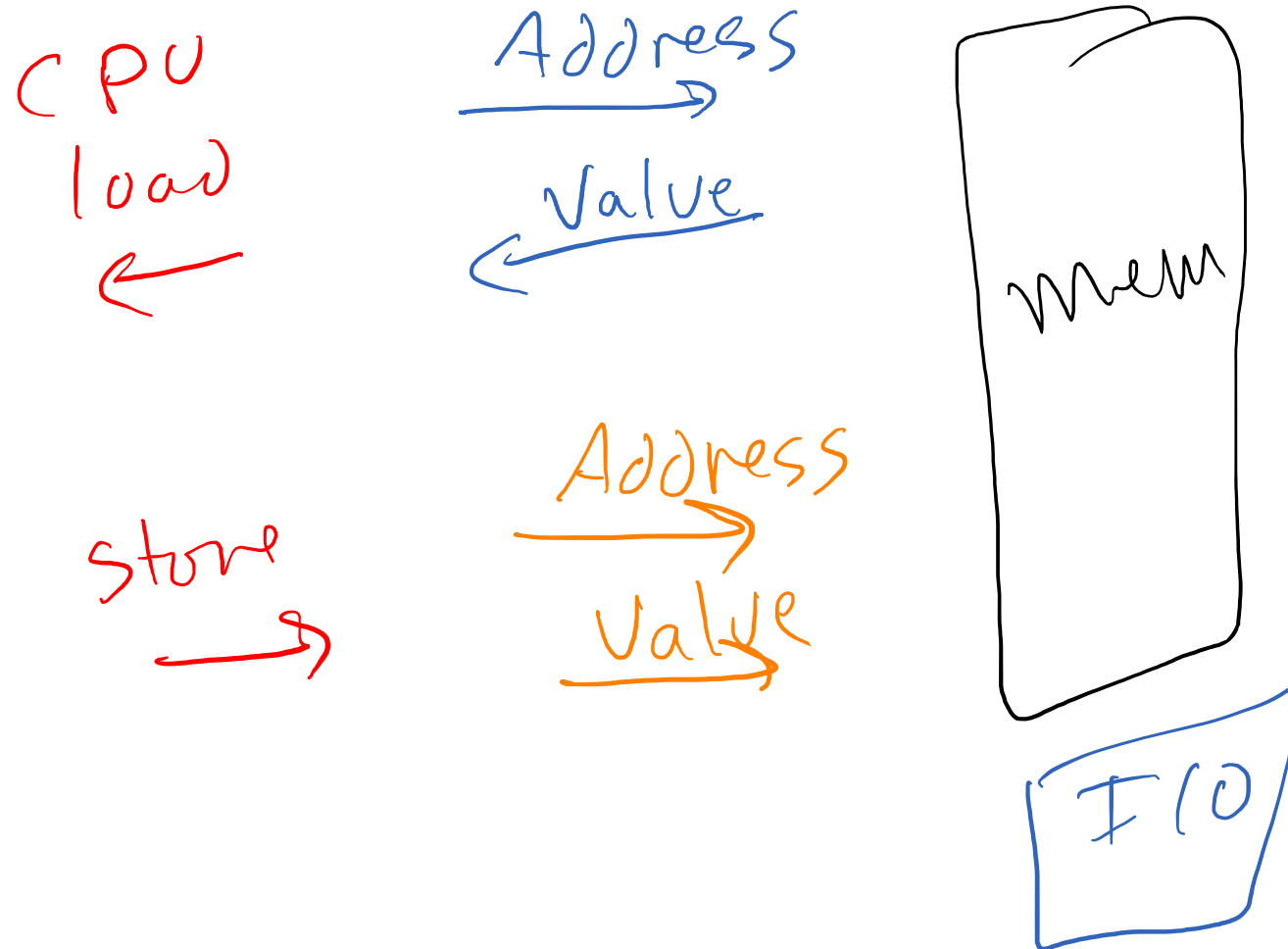
```
arm-none-eabi-gcc -o test.o test.c -g
```

```
-O1 --specs=nosys.specs
```

```
arm-none-eabi-objdump -DS test.o > test.dis
```

What do the CPU and Memory need to communicate?

or I/O



The System Bus

